

CVA6 Sv39 Malformed Non-Leaf PTE Guard Bypass Vulnerability Report

Affected version

Issue package: ../../issues/CV-2

This package was validated on a local CVA6 checkout at revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the standard non-H cv64a6_imafdc_sv39 configuration. No fix revision has been identified yet.

Background

The issue package states the architectural bug: on the non-H cv64a6_imafdc_sv39 profile, CVA6 can accept a malformed Sv39 non-leaf PTE as a next-level table pointer instead of raising a page fault. The malformed entry has V=1, R=W=X=0, and A=1.

The security PoC in this package places a retained readable leaf behind that architecturally faulting root entry, modeling a monitor or sandbox that uses the malformed non-leaf entry as a software guard.

Vulnerability type and CWE

Vulnerability type. Sv39 page-table guard bypass due to missing reserved-bit fault on non-leaf PTEs.

CWE. Primary: CWE-284 Improper Access Control. Secondary: CWE-200 Exposure of Sensitive Information and CWE-693 Protection Mechanism Failure.

Core architectural defect

For Sv39 non-leaf PTEs, the A, D, and U bits are reserved. The translation algorithm requires the walk to stop with a page fault if reserved future-standard bits are set in any PTE used by the walk.

The issue-level primitive is:

```
/* root[1]: malformed non-leaf PTE, V=1/RWX=0/A=1. */
la    t1, l1_pt
srli  t1, t1, 12
slli  t1, t1, 10
ori   t1, t1, (PTE_TABLE | PTE_A)
sd    t1, (ROOT_BAD_IDX * 8)(t0)

load_site:
ld    t1, 0(t0)
```

The lower-level page tables are otherwise valid. A compliant implementation must fault at the malformed root entry and must not reach the lower-level leaf. The issue package shows that CVA6 follows the malformed non-leaf entry and completes the load.

The PTW code has a common invalid-PTE filter, but that filter does not reject A, D, or U on non-leaf PTEs:

```
if (!pte.v || (!pte.r && pte.w) ||
    (!pte.reserved && CVA6Cfg.XLEN == 64) ||
    (!CVA6Cfg.SvnapotEn && pte.n) ||
    (CVA6Cfg.SvnapotEn && !(pte.r || pte.x) && pte.n))
    state_d = PROPAGATE_ERROR;
```

The non-leaf path does contain an explicit A/D/U rejection, but it is guarded by CVA6Cfg.RVH. Reduced to the branches that matter for the tested non-H profile, the block behaves like this:

```
if (ptw_lvl_q[0] == CVA6Cfg.PtLevels - 1) begin
    state_d = PROPAGATE_ERROR;
    if (CVA6Cfg.RVH) ptw_stage_d = ptw_stage_q;
end else begin
    ptw_lvl_n[0] = ptw_lvl_q[0] + 1'b1;
    state_d = WAIT_GRANT;

    if (!CVA6Cfg.RVH) begin
        ptw_pptr_n = {pte.ppn, vaddr_lvl[0][ptw_lvl_q[0]],
                     (CVA6Cfg.PtLevels)'(0)};
    end

    if (CVA6Cfg.RVH && (pte.a || pte.d || pte.u)) begin
        state_d = PROPAGATE_ERROR;
        ptw_stage_d = ptw_stage_q;
    end
end
```

The tested non-H profile disables the hypervisor extension, so this guarded check is inactive:

```
localparam CVA6ConfigHExtEn = 0; // always disabled
```

```
RVH: bit'(CVA6ConfigHExtEn),
```

For the tested profile, non-H Sv39 walks can take the non-leaf continuation path before any pte.a || pte.d || pte.u reserved-bit rejection is applied. The guarded root entry V=1, RWX=0, A=1 is treated as a next-level table pointer instead of as a faulting non-leaf PTE.

Security relevance

The security setting is an M-mode monitor or sandbox manager that retains a lower-level page-table subtree in memory but marks its root entry as an architecturally faulting guard:

```
- V = 1
- R = W = X = 0
- A = 1
```

This encoding is not a valid non-leaf PTE. A compliant Sv39 walk must raise a page fault before reaching the retained subtree. Such a guard can be used to make a subtree temporarily unreachable while preserving lower-level page-table state for auditing, recovery, lazy teardown, or later reactivation.

In a vulnerable CVA6 configuration, the S-mode workload can access the guarded virtual address anyway. CVA6 follows the malformed non-leaf PTE, reaches the retained readable leaf, and exposes the protected data behind the guard.

Security PoC

Assumptions

Trusted M-mode code installs S-mode page tables and expects Sv39 reserved-bit faults to enforce that a guarded subtree is unreachable. The S-mode workload can issue ordinary loads to its virtual address space but cannot modify the monitor's page tables.

PoC setup

The PoC maps normal code/data at 0x80000000 through a valid 1 GiB identity leaf. It then creates a guarded subtree at:

- GUARDED_VA = 0x40000000;
- root guard entry: V=1, RWX=0, A=1;
- retained lower levels: valid table entries;
- retained leaf: readable mapping to guarded_secret;
- SECRET_WORD = 0x7365637265743137;
- initially zero attacker_buffer in normal memory.

The M-mode monitor drops to S-mode at s_tenant_probe. The S-mode workload performs:

```
li    t0, GUARDED_VA
guarded_load_site:
ld    t1, 0(t0)
```

If the load incorrectly returns SECRET_WORD, the workload copies it to attacker_buffer and exits with code 31.

Security result

- The page-table root entry is an architecturally invalid non-leaf guard.
- The lower-level leaf points at a protected word that should be unreachable while the guard is present.
- CVA6 follows the malformed guard and completes the S-mode load.
- The PoC stores the leaked word into attacker_buffer and exits through the leak_guarded_secret path.

CVA6 evidence

The evidence log is poc/poc.cva6.log:

```
poc.elf *** FAILED *** (tohost = 31) after 595 cycles
*** [rvfi_tracer] INFO: Simulation terminated after      583 cycles!
```

tohost = 31 is the PoC's leak_guarded_secret path. That path is reached only after the S-mode load reads SECRET_WORD through the guarded subtree and stores it to attacker_buffer.

Expected result on a compliant core

The S-mode load at guarded_load_site raises a load page fault with mtval = GUARDED_VA. The M-mode trap handler confirms the fault metadata, checks that attacker_buffer is still zero, and exits with code 0.

Observed result on CVA6

CVA6 exits with tohost = 31, meaning the S-mode workload read SECRET_WORD from behind the malformed non-leaf guard and copied it into attacker_buffer.

Impact

- Primary: Confidentiality. Data behind a guarded page-table subtree can be disclosed to an S-mode workload even though the root entry should fault.

- Secondary: Integrity. If monitor policy assumes the guarded subtree is unreachable, CVA6 can execute data-access behavior that violates that policy.

Affected version status

Confirmed affected in a local run of CVA6 revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the cv64a6_imafdc_sv39 configuration.

Fixed: none identified yet.

Fix direction

The Sv39 page-table walker should reject malformed non-leaf PTEs with reserved A, D, or U bits set in all configurations, not only under RVH-specific branches. The walk must stop with a page fault before using such an entry as a next-level table pointer. A minimal fix shape is to apply the non-leaf reserved-bit check before or alongside the next-level pointer update, without gating it on CVA6Cfg.RVH:

```
if (!(pte.r || pte.x) && (pte.a || pte.d || pte.u)) begin
    state_d = PROPAGATE_ERROR;
    if (CVA6Cfg.RVH) ptw_stage_d = ptw_stage_q;
end
```